

I N D E X

NAME: P. Sai Ujwal STD.: 9th SEC.: D ROLL NO.: — SUB.: OS Lab

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
		<u>Week 1</u>		
1)	13/3/26	FCFS Algorithm		<u>A</u> 13/3
		SF Algorithm		<u>A</u> 13/3
2	13/3/26	Priority scheduling cases		<u>A</u> 13/3
3)	27/3/26	Round Robin		<u>A</u> 27/3
		Algorithm & Multi-lvl queue		<u>A</u> 27/3
4)	10/4/26	RMS, EDF ^{prop.} scheduling algorithms.		<u>A</u> 8/4/26
5)	24/4/26	Bounded Buffer Problem		<u>A</u> 24/4
		Dining Philosophers Problem.		
7)	8/5/26	Banker's Algorithm & Deadlock Detection	(10)	<u>A</u> 8/5/26
8	15/5/26	Memory Allocation	(10)	<u>A</u> 15/5
9)	22/5/26	Pages Faults.		<u>A</u> 3/6/26

[illegible]

① FCFS - c program

#code#

#include <stdio.h>

void swap (int *a, int *b) {

int temp = *a;

*a = *b;

*b = temp;

}

int main() {

int n;

float avg_tat = 0, avg_wt = 0;

printf("Enter no. of processes: \n");

scanf("%d", &n);

int P[10], AT[10], BT[10], TAT[10],
CT[10], WT[10];

for (int i = 0; i < n; i++) {

P[i] = i + 1;

printf("Enter the arrival time of
P %d", P[i]);

scanf("%d", &AT[i]);

printf("Enter burst time of
P %d", P[i]);

scanf("%d", &BT[i]);

}

for (int i = 0; i < n; i++) {

for (int j = i + 1; j < n; j++) {

if (AT[i] > AT[j]) {

swap(&AT[i], &AT[j]);

swap(&P[i], &P[j]);

swap(&BT[i], &BT[j]);

}

}


```

CTC[0] = ATC[0] + BTC[0];
for (int i = 1; i < n; i++) {
    if (CTC[i-1] < ATC[i]) {
        CTC[i] = ATC[i] + BTC[i];
    }
    else {
        CTC[i] = CTC[i-1] + BTC[i];
    }
}

```

```

for (int i = 0; i < n; i++) {
    TAT[i] = CTC[i] - ATC[i];
    WTC[i] = TAT[i] - BTC[i];
    avg-tat += TAT[i];
    avg-wt += WTC[i];
}

```

```

avg-tat /= n;
avg-wt /= n;
printf ("In Process t AT t BT t CT t TAT\n");

```

```

for (int i = 0; i < n; i++) {
    printf ("P%d\t P%d\t %d\t %d\t\n",
           PC[i], ATC[i], BTC[i], CTC[i],
           TAT[i], WTC[i]);
}

```

```

printf ("In Average TAT = %.2f\n", avg-tat);
printf ("Average WT = %.2f\n", avg-wt);
return 0;
}

```


output:-

Enter no. of processes: 5

Enter the burst time of P1: 2

Enter the arrival time of P1: 4

Enter the burst time of P2: 3

Enter the ~~burst~~ arrival time of P2: 6

Enter the burst time of P3: 7

Enter the arrival time of P3: 5

Enter the burst time of P4: 4

Enter the arrival time of P4: 8

Enter the arrival time of P5: 3

Enter the burst time of P5: 2

Process	AT	BT	CT	TAT	WT
P1	2	4	6	4	0
P5	3	2	5	5	3
P3	5	7	15	10	3
P2	6	3	18	12	9
P4	8	4	22	14	10

Average TAT = 9.00

Average WT = 5.00

② SJF C both preemptive & non preemptive cases)
code:-

```
#include <stdio.h>
```

```
int main()
```

```
{ int choice;
```

```
printf ("CPU scheduling - SJF\n");
```

```
printf ("1. preemptive SJF\n");
```

```
printf ("2. non preemptive SJF\n");
```

```
printf ("Enter choice:");
```

```
scanf ("%d", &choice);
```

```
switch (choice)
```

```
{ case 1 :
```

```
int n, i, j;
```

```
int bt[20], wt[20], tat[20], p[20];
```

```
float avg_wt = 0;
```

```
float avg_bt = 0;
```

```
printf ("enter burst Time\n");
```

```
for (int i = 0; i < n; i++)
```

```
printf ("%d", i+1);
```

```
scanf ("%d", &bt[i]);
```

```
p[i] = i+1;
```

```
}
```

```
for (i = 0; i < n; i++)
```

```
for (int j = i+1; j < n; j++)
```

```
if (bt[i] > bt[j])
```

```
{ int temp = bt[i];
```

```
bt[i] = bt[j];
```

```
bt[j] = temp;
```

```
temp = p[i];
```



```

PC[i] = PC[j];
PC[j] = temp;
}
}
}
wt[0] = 0;
for (int i = 1; i < n; i++) {
    wt[i] = wt[i-1] + bt[i-1];
}
for (int i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}
printf("In process\t Bt\t wt\t TAT\n");
for (int i = 0; i < n; i++) {
    avg-wt += wt[i];
    avg-tat += tat[i];
    printf("P%d\t Bt %d\t wt %d\t TAT %d\n",
           i, bt[i], wt[i], tat[i]);
}
printf("In Average - WT = %.2f", avg-wt/n);
printf("In Average TAT = %.2f", avg-tat/n);
break;
}
case 2: {
    int n, i, time = 0, smallest;
    int bt[10], rt[10], at[10];
    int complete = 0;
    int wt[10], tat[10];
    float avg-wt = 0, avg-tat = 0;
    printf("Enter no. of processes: ");
    scanf("%d", &n);

```

```

for (int i = 0; i < n; i++) {
    printf("Arrival time of p%d", i+1);
    scanf("%d", &a[i]);
    printf("Burst time of p%d", i+1);
    scanf("%d", &b[i]);
    rt[i] = b[i];
}

```

```

while (complete != n) {

```

```

    smallest = -1;

```

```

    for (int i = 0; i < n; i++) {

```

```

        if (a[i] < time & rt[i] > 0) {
            if (smallest == -1 || rt[i] <
                rt[smallest]) {

```

```

                smallest = i;

```

```

            }

```

```

        }

```

```

        if (smallest == -1) {

```

```

            time++;

```

```

            continue;

```

```

        }

```

```

        rt[smallest]--;

```

```

        time++;

```

```

        if (rt[smallest] == 0) {

```

```

            complete++;

```

```

            tat[smallest] = time - a[smallest];

```

```

            wt[smallest] = tat[smallest] -

```

```

                b[smallest];

```

```

            avg_wt += wt[smallest];

```

```

            avg_tat += tat[smallest];

```

```

        }

```



```

printf("\n Process\t WT\t TAT\n");
for (int i=0; i<n; i++)
    printf("P%02d\t %02d\t %02d\n",
           i+1, wt[i], tat[i]);
}
printf("\n Average WT = %0.2f", avg-wt/n);
printf("\n Average TAT = %0.2f", avg-tat/n);
break;
}
default:
    printf("Invalid choice");
}
return 0;
}

```

output:

Non Preemptive - case "

CPU scheduling - SJF

1) Non preemptive SJF

2) Preemptive SJF (SRTF)

Enter choice : 1

Enter no. of processes : 4

Enter Burst Time :

P1 : 2

P2 : 4

P3 : 3

P4 : 7

Process	BT	WT	TAT
---------	----	----	-----

P1	2	0	2
----	---	---	---

P3	3	2	5
----	---	---	---

P2	4	5	9
----	---	---	---

P4	7	9	16
----	---	---	----

Average WT = 4.00

Average TAT : 8.00

non preemptive case

CPU scheduling - SJF

- 1) Non preemptive SJF
- 2) Preemptive SJF (SRJF)

Enter choice: 2

Enter no. of processes: 4

Arrival Time of P1: 1

Burst Time of P1: 2

Arrival Time of P2: 3

Burst Time of P2: 4

Arrival Time of P3: 5

Burst Time of P3: 6

Arrival Time of P4: 7

Burst Time of P4: 8

Process	WT	TAT
P1	0	2
P2	0	4
P3	2	8
P4	6	14

Average WT = 2.00

Average TAT = 7.00

13/3

TAT	TW	TH	NO. OF
5	0	5	19
2	2	2	29
4	2	4	29
2	2	2	29

00-0 = TW
008: TAT

- Q) Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.
→ Priority (pre-emptive) & (Non-pre-emptive).

#CODE#

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("Priority scheduling Algorithm\n(preemptive)");
```

```
    int n;
```

```
    printf("Enter no. of processes:");
```

```
    scanf("%d", &n);
```

```
    int at[n], bt[n], pr[n], rt[n];
```

```
    int ct[n], tat[n], wtn[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("AT P%d: ", i+1);
```

```
        scanf("%d", &at[i]);
```

```
        printf("BT P%d: ", i+1);
```

```
        scanf("%d", &bt[i]);
```

```
        printf("Priority P%d: ", i+1);
```

```
        scanf("%d", &pr[i]);
```

```
        rt[i] = bt[i];
```

```
}
```

22/8/21

```
int time = 0; complete = 0; // or while (1)
```

```
while (complete < n) {
```

```
    int highest = 999, index = -1;
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        if (arr[i] <= time && arr[i] > 0)
```

```
        {
```

```
            if (arr[i] < highest)
```

```
                highest = arr[i];
```

```
                index = i;
```

```
        }
```

```
    } if (index == -1) {
```

```
        time ++;
```

```
    } continue;
```

```
}
```

```
arr[index] = 0;
```

```
time ++;
```

```
if (arr[index] != 0) {
```

```
    arr[index] = time;
```

```
    complete ++;
```

```
}
```

```
if (index == -1) {
```



```

printf ("INP\n AT BT PR tCT tTAT\n WT\n");
for (int i=0; i<n; i++) {
  tct[i] = ct[i] - at[i];
  wt[i] = tct[i] - bt[i];

  printf ("P %d\t %d\t %d\t %d\t %d\t %d\t\n", i+1, at[i], bt[i], pr[i], ct[i], tct[i], wt[i]);
}

```

Output:

Priority Scheduling Algorithm (Preemptive)
 Enter no. of processes :- 4

- AT P1 : 2
- BT P1 : 4
- Priority P1 : 3
- AT P2 : 7
- BT P2 : 4
- Priority P2 : 2
- AT P3 : 4
- BT P3 : 2
- Priority P3 : 2
- AT P4 : 6 7 8
- BT P4 : 5
- Priority P4 : 4

$\frac{1}{6} \left(\frac{1}{6} + \frac{1}{6} + \frac{1}{6} + \frac{1}{6} + \frac{1}{6} + \frac{1}{6} \right) \frac{1}{6} \frac{1}{6} \frac{1}{6}$
 $\left(\frac{1}{6} + \frac{1}{6} \right) \left(\frac{1}{6} + \frac{1}{6} \right) \left(\frac{1}{6} + \frac{1}{6} \right) \left(\frac{1}{6} + \frac{1}{6} \right)$
 $\left(\frac{1}{6} + \frac{1}{6} \right) \left(\frac{1}{6} + \frac{1}{6} \right) \left(\frac{1}{6} + \frac{1}{6} \right) \left(\frac{1}{6} + \frac{1}{6} \right)$

S: 19 TA
 M: 19 TD
 S: 19 M19019
 E: 19 TA
 M: 19 TD
 S: 19 M19019
 M: 19 TA
 E: 19 TD
 S: 19 M19019
 E: 19 TA
 S: 19 TD
 M: 19 M19019

Non Preemptive

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("Priority Scheduling Algorithm  
          (Non-Preemptive)\n");
```

```
    int n;
```

```
    printf("Enter no. of processes:");
```

```
    scanf("%d", &n);
```

```
    int at[n], bt[n], pr[n], ct[n], tat[n],  
        wt[n];
```

```
    int completed[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("AT %d", i+1);
```

```
        scanf("%d", &at[i]);
```

```
        printf("BT %d", i+1);
```

```
        scanf("%d", &bt[i]);
```

```
        printf("Priority %d", i+1);
```

```
        scanf("%d", &pr[i]);
```

```
        completed[i] = 0;
```

```
    }
```

```
    int tm = 0, done = 0;
```

```
while (done < n)
```

```
{
```

```
    int highest = 999, index = -1;
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        if (at[i] <= time && completed[i] == 0)
```

```
        {
```

```
            if (pr[i] < highest) &
```

```
            { highest = pr[i];
```

```
              index = i;
```

```
            }
```

```
        }
```

```
    }
```

```
    if (index == -1) time++;
```

```
    else {
```

```
        time += bt[index];
```

```
        et[index] = time;
```

```
        complete[index] = 1;
```

```
        done++;
```

```
    }
```

```
}
```

```
printf("\n P \t A \t V \t B \t I \t P \t C \t T \t A \t T \t W \t  
      \n");
```

```
for (int i = 0; i < n; i++) {
```

```
    tat[i] = ct[i] - at[i];
```

```
    wt[i] = tat[i] - bt[i];
```

```
    printf("%d \t %d \t %d \t %d \t  
           %d \t %d \t \n",
```

```
           i+1, at[i], bt[i], pr[i],  
           ct[i], tat[i], wt[i]);
```

```
}
```


OUTPUT:-

Enter no. of Processes: 3

AT P₁: 0BT P₁: 3Priority P₁: 2AT P₂: 2BT P₂: 2Priority P₂: 1AT P₃: 2BT P₃: 3Priority P₃: 3

--- Preemptive Priority Scheduling ---

P	AT	BT	Pr	CT	TAT	WT
P ₁	0	3	2	5	5	2
P ₂	2	2	1	4	2	0
P ₃	2	3	3	8	6	3

Average TAT = 4.00

Average WT = 1.33

13/3

Q) Write a C program to simulate round robin scheduling.

```
#include
```

```
#include <stdio.h>
```

```
struct process
```

```
{
    int pid, at, bt, ct, tat, wt, ret;
};
```

```
int main() {
```

```
    struct process p[20];
```

```
    int queue[100];
```

```
    int front = 0, rear = 0;
```

```
    int time = 0, completed = 0;
```

```
    int visited[20] = {0};
```

```
    float avgwt = 0, avgtat = 0;
```

```
    int tq, n;
```

```
    printf("Enter time quantum: ");
```

```
    scanf("%d", &tq);
```

```
    printf("Enter no of processes: ");
```

```
    scanf("%d", &n);
```



```

for (int i = 0; i < n; i++) {
    printf("Enter arrival time of  
process %d : ", i + 1);
    scanf("%d", &pc[i].bt);

```

```

    pc[i].pid = i + 1;
    pc[i].rt = pc[i].bt;

```

```

}

```

```

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        if (pc[i].at > pc[j].at) {
            struct process temp = pc[j];
            pc[j] = pc[i];
            pc[i] = temp;
        }
    }
}

```

```

time = pc[0].at;

```

```

queue[rear++] = 0;

```

```

visited[0] = 1;

```

```

while (completed < n) {

```

```

    if (front == rear) {

```

```

        time++;

```

```

        for (int i = 0; i < n; i++) {

```

```

            if (pc[i].at <= time & !visited[i]) {

```

```

                queue[rear++] = i;

```

```

                visited[i] = 1;

```

```

            }
        }
        continue;
    }
}

```

```

int idx = queue[front++];
if (p[idx].ret > tq)
    p[idx].ret = tq;
time = tq;

```

```

for (int i = 0; i < n; i++)
    if (p[i].at <= time & !visited[i])

```

```

        queue[rear++] = i;

```

```

        visited[i] = 1;

```

```

        queue[rear++] = idx;

```

```

    }

```

```

else {

```

```

    time = p[idx].ret;

```

```

    p[idx].ret = 0;

```

```

    p[idx].ct = time;

```

```

    p[idx].tat = p[idx].ct - p[idx].at;

```

```

    p[idx].wt = p[idx].tat - p[idx].bt;

```

```

    ag wt += p[idx].wt;

```

```

    arg tat += p[idx].tat;

```

```

    completed++;

```


OUTPUT

Enter time quantum = 3

Enter arrival time of process 1 = 0

Enter burst time = 7

Enter arrival time of process 2 = 3

enter burst time = 5

Enter arrival time of process 3 = 6

Enter burst time = 11

--- Round Robin Scheduling ---

Process / A.T / B.T / C.T / T.A.T / W.T

P1 0 7 12 12 5

P2 3 5 10 7 2

P3 6 11 23 17 6

Average T.A.T = 12.00

Average W.T = 4.33

* If time quantum = 1

Average T.A.T = 14

Average W.T = 6.33

* If time quantum = 3

Average T.A.T = 14.33

Average W.T = 6.67

Analysis: There exists an optimal time quantum where average waiting time is minimal.

- Q) Write a C program demonstrating RMS and EDF scheduling.

#CODE#

```
#include <stdio.h>
#include <math.h>
#define MAX 20
#define MAX_TIME 1000
```

```
struct Task {
    int id, bt, deadline, period, ct, wt, tat;
};
```

```
double utilization_edf(struct Task tasks[],
                       int n) {
```

```
    double u = 0;
```

```
    for(int i = 0; i < n; i++)
```

```
        u += (double) tasks[i].bt / tasks[i].deadline;
```

```
    return u;
```

```
}
```

```
double utilization_RMS(struct Task tasks[],
                       int n) {
```

```
    double u = 0;
```

```
    for(int i = 0; i < n; i++)
```

```
        u += (double) tasks[i].bt / tasks[i].deadline;
```

```
    return u;
```

```
}
```

```
double rm-bound (int n) {
    return n * pow(2.0, 1.0/n) - 1.0;
}
```

```
void edf (struct Task tasks[], int n) {
    int tm = 0;
    int schedule [MAX_TIME] = {0};

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (tasks[i].deadline > tasks[j].deadline) {
                struct Task tmp = tasks[i];
                tasks[i] = tasks[j];
                tasks[j] = tmp;
            }
        }
    }
}
```

```
printf("\n === EDF scheduling ===\n");
printf("%10s %10s %10s %10s %10s %10s\n", "ProcessID", "BT", "Deadline", "CT", "WT", "TAT");
```

```
for (int i = 0; i < n; i++) {
    for (int k = time; k < time + tasks[i].bt; k++) {
        schedule[k] = tasks[i].id;
        time += tasks[i].bt;
        tasks[i].bt = time;
        tasks[i].wt = tasks[i].ct;
    }
}
```



```

tasks[i].wt = tasks[i].tat - tasks[i].bt;
printf (" %d %d %d %d %d\n", tasks[i].id, tasks[i].bt,
tasks[i].deadline, tasks[i].ct, tasks[i].wt,
tasks[i].tat);
}
3

```

```

double u = utilization_cdf (tasks, n);
if (u <= 1.0) printf ("EDF result:
schedulable CU = %2f\n", u);
else
printf ("EDF result, not schedulable
CU = %2f\n", u);

```

```

void rms (struct Task tasks[], int n) {
    int hme = 0;
    int schedule [MAX_TIME] = {0};
}

```

```

for (int i = 0; i < n-1; i++) {
    for (int j = i+1; j < n; j++) {
        if (tasks[i].period > tasks[j].period) {
            struct Task tmp = tasks[i];
            tasks[i] = tasks[j];
            tasks[j] = tmp;
        }
    }
}
3

```

```
printf("\n == Rate Monotonic  
Scheduling == == == \n")
```

```
printf("%10d%-5d%-6d%-5d%-5d%-5d",  
tasks[i].id, tasks[i].bt, tasks[i].per,  
tasks[i].ct, tasks[i].wt, tasks[i].tex);
```

```
}
```

```
double u = utilization_rms(tasks, n);
```

```
double bound = rm_bound(n);
```

```
if (u <= bound) printf("RMS result:  
Schedulable (U = %0.2f, Bound = %0.2f) \n",  
u, bound);
```

```
else printf("RMS Result: Not schedulable  
: (U = %0.2f, Bound = %0.2f) \n",  
u, bound);
```

```
}
```

```
int main() {
```

```
int n;
```

```
printf("Enter no. of processes: ");
```

```
scanf("%d", &n);
```

```
struct Task tasks[n];
```

```
printf("\nEnter process details:\n");
```

```
for (int i = 0; i < n; i++) {
```

```
tasks[i].id = i + 1;
```

```
printf("\n Process %d:\n", i);
```

```
printf("Burst time (for EDF): ");
```

```
scanf("%d", &tasks[i].deadline);
```

```
printf("Period (for RMS): ");
```

```
scanf("%d", &tasks[i].period);
```


3

edt(tasks, n);

rms(tasks, n);

return 0;

3

OUTPUT

(1) enter process details :

Process 0

Burst time (for EDF) : 2

Deadline (for EDF) : 5

Period (for RMS) : 5

Process 1

Burst time (for EDF) : 2

Deadline (for EDF) : 7

Period (for RMS) : 7

Process 2

Burst time (for EDF) : 3

Deadline (for EDF) : 8

Period (for RMS) : 2

~~27~~

==== Earliest Deadline First (EDF) scheduling

Process ID	BT	Deadline	CT	WT	TAT
1	2	5	2	0	2
2	2	7	4	2	4
3	3	8	7	4	7

EDF result : Not schedulable ($U = 1.06$)

==== Rate Monotonic scheduling

Process ID	BT	Deadline	CT	WT	TAT
1	2	5	2	0	2
2	2	7	4	2	4
3	3	8	7	4	7

RMS result : Not schedulable ($U = 1.06$,
Bound = 0.78)

8/4/20

#Week 5#

6

classmate

Date _____
Page _____

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <fcntl.h>
#include <unistd.h>
```

```
#define BUFFER_SIZE 3
#define NUM_ITEMS 15
```

```
int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
sem_t *empty;
sem_t *full;
pthread_mutex_t mutex;
```

```
void *producer(void *arg) {
    for (int item = 0; item < NUM_ITEMS; item++) {
        sem_wait(empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Produced : %d at buffer[%d]\n",
               item, in);
        in = (in + 1) % BUFFER_SIZE;
```

```
pthread_mutex_unlock(&mutex);  
sem_post(&full);
```

```
usleep(1000000);
```

```
}
```

```
return NULL;
```

```
}
```

```
void * consumer(void *arg) {
```

```
    for (int i = 0; i < NUM_ITEMS; i++) {
```

```
        sem_wait(&full);
```

```
        pthread_mutex_lock(&mutex);
```

```
        int item = buffer[out];
```

```
        printf("consumed %d from  
        buffer [%d]\n", item, out);
```

```
        out = (out + 1) % BUFFER_SIZE;
```

```
        pthread_mutex_unlock(&mutex);
```

```
        sem_post(&empty);
```

```
        usleep(150000);
```

```
    return NULL;
```

```
}
```



```
int main(void) {
```

```
pthread_mutex_t mutex;
pthread_create(&prod_tid, NULL,
```

```
producer, NULL);
```

```
pthread_create(&cons_tid, NULL,
```

```
consumer, NULL);
```

```
pthread_join(prod_tid, NULL);
```

```
pthread_join(cons_tid, NULL);
```

```
sem_close(empty);
```

```
sem_close(full);
```

```
pthread_mutex_destroy(&mutex);
```

```
return 0;
```

1 1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

Output:-

3 (1101) 1000 111

produced : 0 at buffer[0]

consumed : 0 from buffer[0]

P : 1 from buffer[0]

C : 2 from buffer[1]

Produced : 2 at buffer[2]

consumed : 2 from buffer[2]

produced : 3 at buffer[0]

C : consumed : 4 at buffer[1]

consumed : 3 from buffer[0]

consumed : 4 from buffer[1]

Trace :

Step	Op.	Buffer[0]	fn	ent	empty	full	mutex
1	Prod → 0	0----	1	0	0	0	1
2	Cons ← 0	-----	1	1	5	0	1
3	Prod → 1	-1---	2	1	4	1	1
4	Cons ← 1	-----	2	2	5	0	1
5	Prod → 2	--2--	3	2	4	1	1
6	Cons ← 2	-----	3	3	5	1	1
7	Prod → 3	---3-	4	3	4	1	1

Dining Philosopher's Problem

classmate

Date _____

Page _____

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <pthread.h>
```

```
#define N 5
```

```
#define EATCOUNT 3
```

```
pthread_mutex_t forks[N];
```

```
void* philosopher(void* arg)
```

```
{ int id = (int) arg;
```

```
for (int i = 0; i < EATCOUNT; i++) {
```

```
    printf("Philosopher %d is thinking.\n", id);
```

```
    if (id % 2 == 0) {
```

```
        pthread_mutex_lock(&forks[id]);
```

```
        pthread_mutex_lock(&forks
```

```
[id + 1] % N);
```

```
    } else {
```

```
        pthread_mutex_lock(&forks[(id + 1) % N]);
```

```
        pthread_mutex_lock(&forks[id]);
```

```
    }
```

```
    printf("Philosopher %d is eating.\n", id);
```

```
    pthread_mutex_unlock(&forks[id]);
```

```
    pthread_mutex_unlock(&forks[(id + 1) % N]);
```

```
    printf("Philosopher %d put down fork.\n", id);
```

```
    return NULL;
```

```

int main() {
    pthread_t philo[N];
    int i;
    for (i = 0; i < N; i++)
        pthread_create(&philo[i], NULL,
            i, i);
}

```

```

for (i = 0; i < N; i++)
    pthread_create(&philo[i], NULL,
        i, i);

```

```

for (i = 0; i < N; i++)
    pthread_create(&philo[i], NULL,
        i, i);

```

```

return 0;

```

3

2

1

0

-1

-2

-3

-4

-5

-6

-7

-8

-9

-10

output.

philosopher	0	is thinking
	0	is eating
	3	is thinking
	3	is eating
	3	put down forks
	3	is thinking
	4	is thinking
	4	is eating
	4	put down forks
	1	is thinking
	1	is eating
	1	put down forks
	2	is thinking
	2	is eating
	2	put down forks

#Week 7#

Inc

.

- Q) Write a C program to demonstrate
- 1) Banker's Algorithm
 - 2) Deadlock Detection

```
#include <stdio.h>
#include <stdbool.h>
```

```
void calculateNeed (int n, int m, int max[n][m],
    int alloc[n][m], int need[n][m]) {
```

```
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            need[i][j] = max[i][j] -
                alloc[i][j];
        }
    }
```

}

```
bool isSafe (int n, int m, int alloc[n][m],
    int need[n][m], int avail[m]) {
    bool finish[n];
    for (int i = 0; i < n; i++) finish[i] = false;
```



```

int safeSeq[n], work[m];
for (int j = 0; j < m; j++) work[j] = avail[j];

```

```

int count = 0;
while (count < n) {
    bool found = false;
    for (int i = 0; i < n; i++) {
        if (!finish[i]) {
            int j;
            for (j = 0; j < m; j++) {
                if (need[i][j] > work[j]) break;
            }
            if (j == m) {
                for (int k = 0; k < m; k++) {
                    work[k] += alloc[i][k];
                }
                safeSeq[count++] = i;
                finish[i] = true;
                found = true;
            }
        }
    }
    if (!found) {
        printf("System not in safe state");
        return false;
    }
}

```

3

```

printf("In System is in safe state ; In safe sequence is");

```

```

for (int i = 0 ; i < m ; i++) {
    if (request[i] > need[process[i]]) {
        printf("Error: Process has
        exceeded max claim.\n");
        return false;
    }
}

```

```

for (int i = 0 ; i < m ; i++) {
    if (request[i] > avail[i]) {
        printf("Resources not
        available. Process must
        wait.\n");
        return false;
    }
}

```

```

for (int i = 0 ; i < m ; i++) {
    avail[i] -= request[i];
    alloc[process[i]] += request[i];
    need[process[i]] -= request[i];
}
return false;

```



```

int main() {
    int n, m;
    printf("Enter no. of processes: ");
    scanf("%d", &n);
    printf("Enter no. of resources: ");
    scanf("%d", &m);

```

```

    int alloc[n][m], max[n][m], avail[m],
    need[n][m];

```

```

    printf("Enter Allocation Matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);

```

Maximum demand.

```

    printf("Enter Available Resource:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);

```

```

        }
    }
    printf("Enter Available resources:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &avail[i][j]);

```

```

        }
    }

```

calculateNeed(n, m, max, alloc, need);

isSafe(n, m, alloc, need, avail);

int *request[m] = {1, 0, 2};

~~printf~~ C++

requestResources(1, n, m, request,
alloc, need, avail);

return 0;

}

Output:-

Enter no. of processes: 3

Enter no. of resources: 3

Enter Allocation Matrix:

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter Maximum Demand Matrix:

7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

Enter Available Resources:

3 3 2

system is in safe state.

safe sequence is : $P_1 \rightarrow P_3 \rightarrow P_4 \rightarrow P_0 \rightarrow P_2$

Deadlock Detection

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
void allocateNeed (int n, int m, int max[n][m],  
                  int alloc[n][m], int need[n][m])
```

```
{
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < m; j++) {
```

```
            need[i][j] = max[i][j] - alloc[i][j];
```

```
        }
```

```
    }
```

```
}
```

```
void detectDeadlock (int n, int m,  
                    int alloc[n][m], int need[n][m], int avail)
```

```
{
```

```
    bool finish[n];
```

```
    for (int i = 0; i < n; i++) finish[i] = false;
```

```
    int work[m];
```

```
    for (int j = 0; j < m; j++) work[j] =  
        avail[j];
```

```
    int count = 0;
```



```

while count < n <
    bool found = false;
    for (int i = 0; i < n; i++) <
        if (!finish[i]) <
            int j;
            for (j = 0; j < m; j++)
                if (need[i][j] > work[j]) ;
                    break;
            if (j == m) <
                for (int k = 0; k < m; k++)
                    work[k] += alloc[i][k] <
                    finish[i] = true;
                    found = true;
                    count++;
            }
        }
    }
    if (!found) break;
}

```

```

bool deadlock = false;
for (int i = 0; i < n; i++) <
    if (!finish[i]) <
        deadlock = true;
        printf ("Process P%d in\n", i);
        printf ("deadlock.\n", i);
    }
}
if (!deadlock)
    printf ("\n No deadlock detected.\n");
}

```

```
int main() {  
    int n, m;  
    printf("Enter no. of processes: ");  
    scanf("%d", &n);  
    printf("Enter no. of resources: ");  
    scanf("%d", &m);
```

```
    int alloc[n][m], max[n][m], avail[m],  
        need[n][m];
```

```
    printf("\nEnter Allocation Matrix: ");
```

```
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < m; j++) {  
            scanf("%d", &alloc[i][j]);  
        }  
    }
```

```
    printf("\nEnter maximum Demand Matrix: ");
```

```
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < m; j++) {  
            scanf("%d", &max[i][j]);  
        }  
    }
```

```
    printf("\nEnter available Resources: ");
```

```
    for (int j = 0; j < m; j++) {  
        scanf("%d", &avail[j]);  
    }
```


calculateNeed (n, m, max, alloc, need) {

printf("In == Deadlock Detection == \n");

DetectDeadlock (n, m, alloc, need, avail);

return 0;

}

Output:

Enter no. of processes: 5

Enter no. of resources: 3

Enter Allocation Matrix:

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter Maximum Demand Matrix:

7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

Enter Available Resources:

3 3 2

== Dead lock Detection ==

NO deadlock detected.

Q) Write a C program to simulate the following memory allocation techniques.

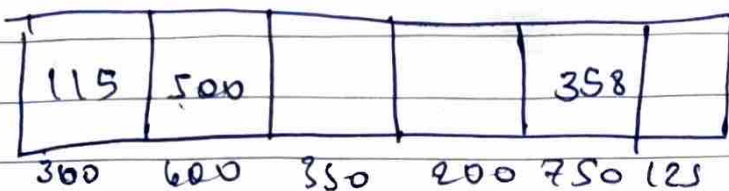
1) Worst Fit

2) Best Fit

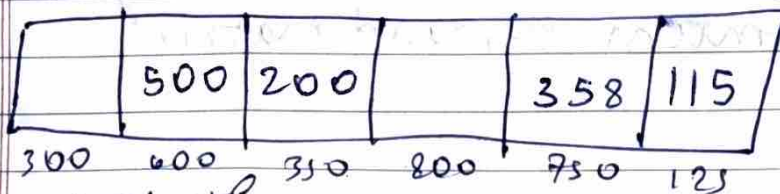
3) First Fit

Given, 6 memory partitions of 300K, 600K, 350K, 200K, 750K, 125K (in order)
how would each of 1st, Best, worst fit algorithms place the processes of 115K, 500K, 358K, 200K, 375K (in order)

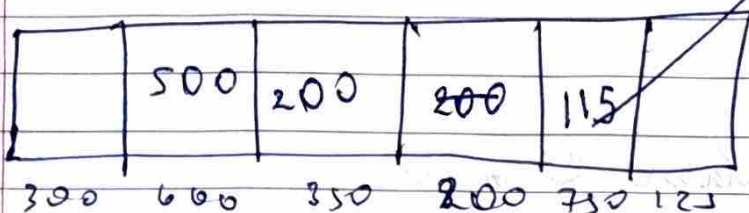
First Fit



Best Fit



Worst Fit



#CODE#

#include <stdio.h>

void firstFit (int blockSize[], int m, int processSize[],
int n)

int allocation[n];

for (int i = 0; i < n; i++) allocation[i] = -1;

for (int i = 0; i < n; i++)

for (int j = 0; j < m; j++)

if (blockSize[j] >= processSize[i])

allocation[i] = j;

blockSize[j] = -1;

break;

}

}

printf("\n --- First Fit --- \n");

printf("Process No. \t Process Size \t
Block No. \n");

for (int i = 0; i < n; i++)

if (allocation[i] != -1)

printf("%d \t %d \t %d \n",
i+1, processSize[i], processSize[i]);

else

printf("%d \t %d \t %d \t Not Allocated \n",
i+1, processSize[i]);

}

}

```

void bestFit (int blockSize[], int m,
    int processSize[], int n) {
    int allocation[n];
    for (int i = 0; i < n; i++) allocation[i] = -1;

    for (int i = 0; i < n; i++) {
        int bestIdx = -1;
        for (int j = 0; j < m; j++) {
            if (blockSize[j] >= processSize[i]) {
                if (bestIdx == -1 || blockSize[j] <
                    blockSize[bestIdx]) {
                    bestIdx = j;
                }
            }
        }
        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            blockSize[bestIdx] = -1;
        }
    }

    printf("\n -- Best Fit -- \n");
    printf("Process No. \t Process Size \t\n\n");
    printf("BLOCK No. \n");

    for (int i = 0; i < n; i++) {
        if (allocation[i] != -1)
            printf("%d \t %d \t %d \n", i+1, processSize[i], allocation[i]);
    }
}

```



```
printf("0%d\t\t0%d\t\tNot Allocation\n",  
i+1, processSize[i]);
```

2. $1.00 \times 10^3 \text{ kg} \cdot \text{m}^3 \cdot \text{s}^{-1}$

3 constant b/c of cost reduction

```
void worstFit (int blockSize[], int m,
              int processSize[], int n) {
    int allocation[n];
```

```
for (int i = 0; i < n; i++) {
```

```
int worstIdx = -1;
```

for $(int\ j = 0; j < m; j++)$ $\{$

```

if (blockSize <= processSize[i])
    if (worstIdx == -1 || blockSize[j]
        > blockSize[worstIdx])

```

WorstIdx = j;

3. $\frac{1}{2} \times \frac{3}{4} = \frac{3}{8}$

3

```
printf ("In --- Worst Fit --- \n");
```

```
printf ("Process No: %d Process Size: %d\n",  
       BlockSize, i * blockSize);
```

for (int i = 0; i < n; i++)

if allocation [?] = -1

```
printf("old %d\t\t\t\t\t%d\n", i+1, processSize[i], allocation[i] + 1);
```

eme

```
print("%d %d %d %d %d %d %d %d %d %d")
```

Not Allocated In", $i+1$, process[id],

3 3

```
int main() {
```

```
    int m, n;
```

```
    printf("Enter no. of memory blocks: ");
```

```
    scanf("%d", &m);
```

```
    int blockSize[m];
```

```
    printf("Enter sizes of %d memory blocks : ", m);
```

```
    for (int i = 0; i < m; i++)
```

```
        scanf("%d", &blockSize[i]);
```

```
    printf("Enter no. of processes: ");
```

```
    scanf("%d", &n);
```

```
    int processSize[n];
```

```
    printf("Enter sizes of %d processes: ");
```

```
    for (int i = 0; i < n; i++) scanf("%d",
```

```
        &processSize[i]);
```

```
    int blockCopy[m];
```

```
    for (int i = 0; i < m; i++)
```

```
        blockCopy[i] = blockSize[i];
```

```
}
```

```
    firstFit(blockCopy, m, processSize, n);
```

```
    bestFit(blockCopy, m, processSize, n);
```

```
    worstFit(blockCopy, m, processSize, n);
```

```
    return 0;
```

```
}
```


Output:-

Enter no. of processes: 5

Enter sizes of 5 processes: 115 500 358 200 375

--- First Fit ---

Pno	Psize	Blk No
1	115	1
2	500	2
3	358	5
4	200	3
5	375	Not Allocated

--- Best Fit ---

Pno	Psize	Blk No
1	115	6
2	500	2
3	358	5
4	200	4
5	375	Not Allocated

--- Worst Fit ---

Process No :	Process size	Block No
1	115	5
2	500	2
3	358	Not Allocated
4	200	3
5	375	Not Allocated.

gk

a) write a C program to simulate page replacement algorithms

a) FIFO

b) LRU

c) Optimal

#CODE#

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void printFrames (int frames[], int m) {
    for (int i = 0; i < m; i++) {
        if (frames[i] == -1) printf("-");
        else printf("%d", frames[i]);
    }
    printf("\n");
}
```

```
int findOptimal (int pages[], int n, int
frame[], int m, int index) {
    int farthest = index, pos = -1;
    for (int i = 0; i < m; i++) {
        int j;
        for (j = index; j < n; j++) {
            if (frame[i] == pages[j]) {
                if (j > farthest) {
                    farthest = j;
                }
                pos = i;
                break;
            }
        }
    }
}
```



```
if (j == n) return i;
```

```
3
```

```
return (pos == -1) ? 0 : pos;
```

```
3
```

```
void fifo (int pages[], int n, int m) {
    int frame[m], count = 0, pageFaults = 0;
    for (int i = 0; i < n; i++) frame[i] = -1;
```

```
    printf("\n --- FIFO simulation --- \n");
```

```
    for (int i = 0; i < n; i++) {
```

```
        int flag = 0;
```

```
        for (int j = 0; j < m; j++) {
```

```
            if (frame[j] == pages[i]) {
```

```
                flag = 1;
```

```
                break;
```

```
        }
```

```
    }
```

```
    if (!flag) {
```

```
        frame[count] = pages[i];
```

```
        count = (count + 1) % m;
```

```
        pageFaults++;
```

```
    }
```

```
    printf("Page %d → ", pages[i]);
```

```
    printfFrames(frame, m);
```

```
3
```

```
printf("Total FIFO Page Faults : %d \n",
    pageFaults);
```

```
3
```

```

void lru (int page[], int n, int m) {
    int frame[m], lastUsed[m], pageFaults = 0;
    for (int i = 0; i < m; i++) {
        frame[i] = -1; lastUsed[i] = -1;
    }

```

```

    printf("\n --- LRU Simulation --- \n");

```

```

    for (int i = 0; i < n; i++) {
        int flag = 0;
        for (int j = 0; j < m; j++) {
            if (frame[j] == page[i]) {
                flag = 1;
                lastUsed[j] = i;
                break;
            }

```

```

        }
        if (!flag) {
            int min = 0;
            for (int j = 1; j < m; j++) {
                min = j;
                break;
            }

```

```

            frame[min] = page[i];
            lastUsed[min] = i;
            pageFaults++;

```

```

        }
        printf("Page %d -> ", page[i]);
        printFrames (frame, m);

```



```

void optimal (int pages[], int n, int m) {
    int frame[m], pageFaults = 0;
    for (int i = 0; i < m; i++) frame[i] = -1;

    printf("\n --- Optimal simulation ---\n");
    for (int i = 0; i < n; i++) {
        int tag = 0;
        for (int j = 0; j < n; j++) {
            int tag = 0;
            for (int j = 0; j < m; j++) {
                if (frame[j] == pages[i]) {
                    tag = 1;
                    break;
                }
            }
            if (!tag) {
                int pos = -1;
                for (int j = 0; j < m; j++) {
                    if (frame[j] == -1)
                        pos = j; break;
                }
                if (pos == -1) pos =
                    findOptimal(pages, n, frame,
                                m, i+1);
                frame[pos] = pages[i];
                pageFaults++;
            }
        }
        printf("Page %d -> ", pages[i]);
        printFrames(frame, m);
    }
}

```

```
int main() {
```

```
    int n, m, choice;
```

```
    printf("Enter number of pages : ");
```

```
    scanf("%d", &n);
```

```
    int pages[n];
```

```
    printf("Enter the reference string : ");
```

```
    for (int i = 0; i < n; i++)
```

```
        scanf("%d", &pages[i]);
```

```
    printf("Enter no. of frames : ");
```

```
    scanf("%d", &m);
```

```
    printf("\n Choose Algorithm : \n");
```

```
    printf("1. FIFO \n 2. LRU \n 3. Optimal \n
```

```
    Enter choice : ");
```

```
    scanf("%d", &(choice));
```

```
    switch (choice) {
```

```
        case 1: fifo(pages, n, m); break;
```

```
        case 2: lru(pages, n, m); break;
```

```
        case 3: optimal(pages, n, m); break;
```

```
        default: printf("Invalid choice \n");
```

```
    }
```

```
    return 0;
```

```
}
```


O/P

Enter no. of pages : 19

Enter the pg reference string?

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 7 0 1

Enter no. of frames k : 4

Choose page Algorithm :

1) FIFO

2) LRU

3) Optimal

1

page 7 → 7 - - -

page 0 → 7 0 - -

page 1 → 7 0 1 -

page 2 → 7 0 1 2

page 0 → 7 0 1 2

page 3 → 3 0 1 2

page 0 → 3 0 1 2

page 4 → 3 4 1 2

page 2 → 3 4 1 2

page 3 → 3 4 1 2

page 0 → 3 4 0 2

page 3 → 3 4 0 2

page 2 → 3 4 0 2

~~page 1 → 3 7 0 1~~~~page 2 → 2 4 0 1~~~~page 0 → 2 4 0 1~~~~page 7 → 2 7 0 1~~~~page 0 → 2 7 0 1~~~~page 1 → 2 7 0 1~~